Best Model Overview

Ingreso predicho por persona + LightGBM | Equipo 06, MECA 4107

Universidad de los Andes

[pendiente]

F1 público en Kaggle

0,782

F1 en holdout, IC 95 % [0,774; 0,789]

0,776

F1 en CV repetida (15 evaluaciones)

Especificación

- **Variable clave:** el ingreso de cada persona, predicho con lo que el test sí trae y sumado al hogar (1.324 variables en total).
- **Algoritmo:** LightGBM (76 %) mezclado con dos redes (19 %) y un rediseño desde cero (5 %); pesos elegidos fuera de muestra.
- **Regla:** pobre si $P(\text{pobre}) \geq 0{,}36$.

Mensajes

1. Las variables aportaron **9 veces más** F1 que los algoritmos.
2. El techo ($\approx 0{,}78$) viene sobre todo de la información que falta.
3. Excluye al 18 % de los pobres, casi todos cerca de la línea.

F1: media armónica de precisión y recall de la clase pobre. Holdout: 20 % del train nunca usado para decidir. Fuente: views/tablas/nivel7/etapa18_resumen.csv.



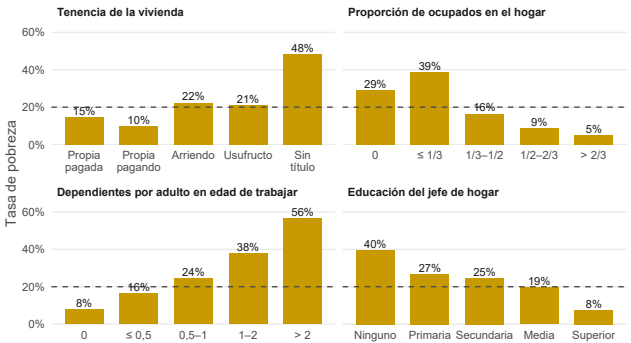
Qué decir (1 min). Nuestro mejor modelo clasifica hogares pobres con F1 de 0,782 en datos que nunca usamos para decidir. La pieza clave no es el algoritmo sino una variable: predecimos cuánto gana cada persona con lo que el test sí trae (ocupación, horas, cotización) y lo sumamos al hogar. Tres mensajes: las variables pesan 9 veces más que el algoritmo, hay un techo que viene de los datos y los errores se concentran cerca de la línea.

Pregunta difícil: ¿por qué una mezcla y no un solo modelo? La mezcla gana +0,0007 en CV repetida frente a LightGBM solo. El IC ingenuo [+0,0002; +0,0012] excluye cero; el corregido por dependencia entre folds no. Es una mejora marginal: en la práctica LightGBM solo rinde igual.

La pobreza responde a umbrales e interacciones: terreno favorable para los árboles

Universidad de los Andes

Tasa de pobreza en train (164.960 hogares). Línea punteada: promedio (20,0%)

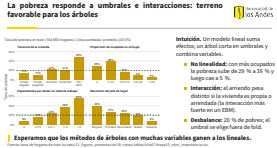


Intuición. Un modelo lineal suma efectos; un árbol corta en umbrales y combina variables.

- **No linealidad:** con más ocupados la pobreza sube de 29 % a 39 % y luego cae a 5 %.
- **Interacción:** el arriendo pesa distinto si la vivienda es propia o arrendada (la interacción más fuerte en un EBM).
- **Desbalance:** 20 % de pobres; el umbral se elige fuera de fold.

Esperamos que los métodos de árboles con muchas variables ganen a los lineales.

Fuente: base de hogares de train (scripts/13_figuras_presentacion.R); views/tablas/nivel7/etapa15_ebm_importancia.csv.



Qué decir. Antes de comparar, una predicción desde los datos. La pobreza no responde de forma lineal: con cero ocupados es menor que con un tercio, porque hay pensionados. Y las variables interactúan: pagar arriendo no significa lo mismo en una casa propia que en una arrendada. Eso favorece a los árboles, que cortan en umbrales y combinan variables sin que se las especifiquemos.

Pregunta difícil: ¿no basta con interacciones en el Logit? Se podrían agregar, pero habría que elegirlas a mano entre más de mil pares posibles; los árboles las buscan solos. El costo es la interpretabilidad.

Cancha nivelada: mismas 48 variables, misma receta y misma validación

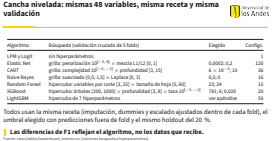
Universidad de los Andes

Algoritmo	Búsqueda (validación cruzada de 5 folds)	Elegido	Configs.
LPM y Logit	sin hiperparámetros		1
Elastic Net	grilla: penalización $10^{[-5,\,0]}$ × mezcla L1/L2 [0, 1]	0,0002; 0,2	120
CART	grilla: complejidad $10^{[-5,\,-1]}$ × profundidad [3, 15]	6×10^{-5}; 10	36
Naive Bayes	grilla: suavizado [0,5; 1,5] × Laplace [0, 3]	0,5; 0	16
Random Forest	hipercubo: variables por corte [3, 25] × tamaño de hoja [5, 60]	23; 24	15
XGBoost	hipercubo: árboles [200, 1000] × profundidad [3, 8] × tasa $10^{[-3,\,-1]}$	781; 6; 0,020	20
LightGBM	hipercubo de 7 hiperparámetros	ver apéndice	50

Todos usan la misma receta (imputación, dummies y escalado ajustados dentro de cada fold), el umbral elegido con predicciones fuera de fold y el mismo holdout del 20 %.

Las diferencias de F1 reflejan el algoritmo, no los datos que recibe.

Fuente: views/tablas/leaderboard_interno.csv (columnas busqueda e hiperparametros).



Cancha nivelada: mismas 48 variables, misma receta y misma validación

Universidad de los Andes

Algoritmo	Búsqueda (validación cruzada de 5 folds)	Elegido	Configs.
LPM y Logit	sin hiperparámetros	—	1
Elastic Net	grilla: penalización $10^{[-5,-1]}$ × mezcla $\{1, 0.5, 0\}$	0,0002; 0,5	120
CART	grilla: complejidad $10^{[-5,-1]}$ × profundidad {1, 15}	4×10^{-5}; 10	36
Naive Bayes	grilla suavizado {0,5; 1,5} × Laplace {0, 1}	0,5; 0	16
Random Forest	hipercubo: variables por corte {1, 25} × tamaño de hoja {5, 40}	23; 24	15
XGBoost	hipercubo: árboles {200, 1000} × profundidad {1, 6} × tasa $10^{[-3,-1]}$	781; 6; 0,020	20
LightGBM	hipercubo de 7 hiperparámetros	ver apéndice	50

Todos usan la misma receta (imputación, dummies y escalado ajustados dentro de cada fold), el umbral elegido con predicciones fuera de fold y el mismo holdout del 20 %.

Las diferencias de F1 reflejan el algoritmo, no los datos que recibe.

Fuente: ranking_holdout.csv y tuning_resultados.csv (columnas de agregados e hiperparámetros).

Qué decir. Para que la comparación sea justa, todos los algoritmos reciben las mismas 48 variables, el mismo preprocesamiento ajustado solo con los datos de entrenamiento de cada fold, y eligen hiperparámetros con validación cruzada de 5 folds. Grilla regular cuando hay 1 o 2 hiperparámetros, hipercubo latino cuando hay más.

Pregunta difícil: ¿no tienen ventaja los que recibieron más configuraciones? LightGBM probó 50 y XGBoost 20, pero XGBoost ya llegaba a 0,746 frente a 0,749: la ventaja de tuning es de 0,003. Elastic Net probó 120 y empata con el Logit sin tuning.

Con las mismas variables, el boosting gana y Naive Bayes queda 11 puntos atrás

Universidad de los Andes

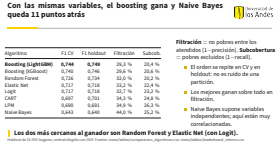
Algoritmo	F1 CV	F1 holdout	Filtración	Subcob.
Boosting (LightGBM)	**0,744**	**0,749**	29,3 %	20,4 %
Boosting (XGBoost)	0,740	0,746	29,6 %	20,6 %
Random Forest	0,726	0,734	32,0 %	20,2 %
Elastic Net	0,717	0,718	33,2 %	22,4 %
Logit	0,717	0,718	32,7 %	23,2 %
CART	0,697	0,701	34,3 %	24,8 %
LPM	0,690	0,691	34,9 %	26,3 %
Naive Bayes	0,643	0,640	44,0 %	25,2 %

Filtración = no pobres entre los atendidos (1−precisión). **Subcobertura** = pobres excluidos (1−recall).

- El orden se repite en CV y en holdout: no es ruido de una partición.
- Los mejores ganan sobre todo en filtración.
- Naive Bayes supone variables independientes; aquí están muy correlacionadas.

Los dos más cercanos al ganador son Random Forest y Elastic Net (con Logit).

Holdout de 32.993 hogares; umbral elegido con OOF. Fuente: views/tablas/comparacion_algoritmos.csv; views/tablas/leaderboard_interno.csv.



Qué decir. Esta es la comparación de los siete algoritmos del curso con las mismas variables. El boosting queda primero y Naive Bayes último. El orden es el mismo en validación cruzada y en holdout. Los dos contendores más cercanos son Random Forest y Elastic Net, que empata con el Logit.

Pregunta difícil: ¿son significativas estas diferencias? Con 32.993 hogares, el efecto mínimo detectable en holdout está entre 0,0025 y 0,006 según el par: las brechas de 0,015 o más están claramente por encima; la de XGBoost frente a LightGBM, no.

Random Forest cubre igual que el boosting, pero filtra más

Universidad de los Andes

Holdout, 48 variables	Random Forest	XGBoost
Recall (cobertura)	0,798	0,794
Precisión	0,680	0,704
No pobres incluidos	2.483	2.206
Pobres excluidos	1.332	1.358
F1	0,734	0,746

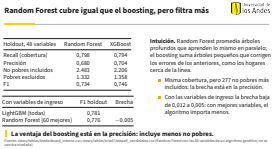
Con variables de ingreso	F1 holdout	Brecha
LightGBM (todas)	0,781	
Random Forest (60 mejores)	0,776	−0,005

Intuición. Random Forest promedia árboles profundos que aprenden lo mismo en paralelo; el boosting suma árboles pequeños que corrigen los errores de los anteriores, como los hogares cerca de la línea.

- Misma cobertura, pero 277 no pobres más incluidos: la brecha está en la precisión.
- Con las variables de ingreso la brecha baja de 0,012 a 0,005: con mejores variables, el algoritmo importa menos.

La ventaja del boosting está en la precisión: incluye menos no pobres.

Fuente: views/tablas/leaderboard_interno.csv; views/tablas/nivel7/etapa9_candidatos.csv (Random Forest con las 60 variables de un algoritmo genético; no es cancha nivelada).



Random Forest cubre igual que el boosting, pero filtra más

Universidad de los Andes

Holdout, 49 variables	Random Forest	XGBoost
Recall (cobertura)	0,786	0,794
Precisión	0,680	0,704
No pobres incluidos	2.483	2.206
Pobres excluidos	1.332	1.358
F1	0,734	0,746

Con variables de ingreso	F1 holdout	Brecha
LightGBM (todas)	0,781	—
Random Forest (60 mejores)	0,776	−0,005

La ventaja del boosting está en la precisión: incluye menos no pobres.

Fuente: cómputos propios (Random Forest, 200 árboles; XGBoost, 1.000 rondas de boosting; LightGBM, todas las variables) sobre la muestra de holdout (20% de la muestra estratificada).

Intuición. Random Forest promedia árboles profundos que aprenden lo mismo en paralelo; el boosting suma árboles pequeños que corrigen los errores de los anteriores, como los hogares cerca de la línea.

- Misma cobertura, pero 277 no pobres más incluidos: la brecha está en la precisión.
- Con las variables de ingreso la brecha baja de 0,012 a 0,005: con mejores variables, el algoritmo importa menos.

Qué decir. Random Forest es el contendor más cercano. Cubre a los mismos pobres que el boosting, pero incluye 277 no pobres más. Una lectura: el bosque promedia árboles independientes, mientras el boosting agrega árboles enfocados en los errores que quedan, que están cerca de la línea. Con las variables de ingreso, la brecha se reduce a la mitad.

Pregunta difícil: ¿la comparación del nivel 7 es justa? No del todo: el bosque usó 60 variables y LightGBM todas. Por eso solo la usamos para mostrar que la brecha se achica, no para medirla.

Elastic Net empata con el Logit: la brecha con el boosting es por la linealidad

Intuición. Elastic Net penaliza coeficientes para evitar el sobreajuste. Con 131.967 hogares y 48 variables no hay sobreajuste que evitar.

- La validación cruzada elige una penalización casi nula (0,0002).
- Mismo F1 en holdout que el Logit: 0,718.
- LPM (probabilidad lineal): 0,691, es decir 0,027 menos que el Logit.

Lo que cuesta la linealidad frente al boosting

- F1: 0,718 frente a 0,749 (−0,031).
- Subcobertura 22,4 % frente a 20,4 %: unos 130 pobres más excluidos en el holdout.
- A cambio, cada coeficiente se puede leer y auditar.

Regularizar no ayuda con muchos datos; lo que falta son umbrales e interacciones.

Fuente: views/tablas/leaderboard_interno.csv (n4_elastic_net, n3_logit, n3_lpm, n6_lightgbm).



Qué decir. El segundo contendor es Elastic Net. La validación cruzada elige casi no penalizar, porque con más de 130 mil hogares y 48 variables no hay sobreajuste. Por eso empata con el Logit. Lo que lo separa del boosting no es la regularización sino la forma funcional: no capta umbrales ni interacciones.

Pregunta difícil: ¿cuántos pobres más excluye? Con 6.605 pobres en el holdout, 2 puntos de subcobertura son unos 130 hogares (1.478 frente a 1.347 excluidos).

Balancear las clases no mejora: solo mueve el umbral

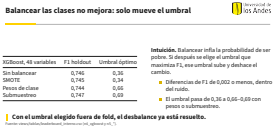
XGBoost, 48 variables	F1 holdout	Umbral óptimo
Sin balancear	0,746	0,36
SMOTE	0,745	0,34
Pesos de clase	0,744	0,66
Submuestreo	0,747	0,69

Intuición. Balancear infla la probabilidad de ser pobre. Si después se elige el umbral que maximiza F1, ese umbral sube y deshace el cambio.

- Diferencias de F1 de 0,002 o menos, dentro del ruido.
- El umbral pasa de 0,36 a 0,66–0,69 con pesos o submuestreo.

Con el umbral elegido fuera de fold, el desbalance ya está resuelto.

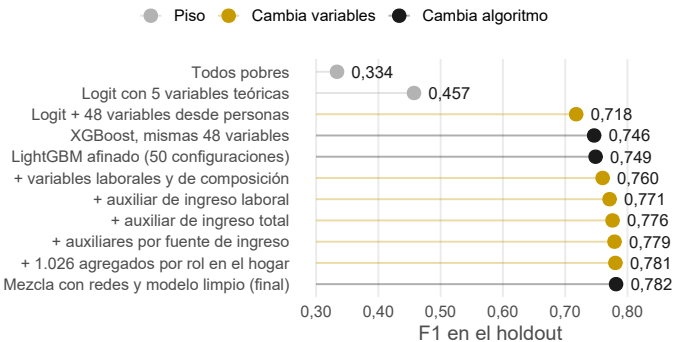
Fuente: views/tablas/leaderboard_interno.csv (n4_xgboost y n5_*).



Qué decir. Probamos las tres estrategias clásicas sobre el mejor algoritmo de ese nivel. Ninguna cambia el F1 más allá de 0,002. Lo que sí cambia es el umbral óptimo, porque el balanceo solo reescala las probabilidades.

Pregunta difícil: ¿y si no se eligiera el umbral? Con el umbral fijo en 0,5, esperaríamos que balancear sí ayudara, porque acerca las probabilidades al punto de corte (no lo medimos). Elegir el umbral es la forma directa de lograr lo mismo.

Cambiar las variables vale 9 veces más que cambiar el algoritmo



Suma de los saltos

- **Variables**: +0,292.
- **Algoritmo**: +0,032.

La variable que más aportó predice el ingreso de cada persona con un modelo auxiliar y lo suma al hogar.

La mejor inversión fue entender el hogar, no elegir el algoritmo.

F1 en holdout. Fuente: views/tablas/leaderboard_interno.csv y nivel7/ (ablacion_etapa1, resumen_etapa2, etapa6_ablacion, etapa7_a, etapa18_resumen).



Qué decir. Poniendo la comparación en perspectiva: toda la diferencia entre algoritmos que acabamos de ver suma 0,03 de F1. Construir variables desde las personas, y sobre todo predecir el ingreso de cada persona, sumó 0,29.

Pregunta difícil: ¿no depende del orden de los pasos? Sí, la descomposición es secuencial. Pero en cualquier orden el contraste es grande: con LightGBM fijo las variables nuevas suman 0,032, más que toda la diferencia entre Logit y boosting.

Con las variables de ingreso, todos los algoritmos convergen

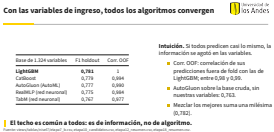
Base de 1.324 variables	F1 holdout	Corr. OOF
LightGBM	**0,781**	1
CatBoost	0,779	0,994
AutoGluon (AutoML)	0,777	0,990
RealMLP (red neuronal)	0,775	0,984
TabM (red neuronal)	0,767	0,977

Intuición. Si todos predicen casi lo mismo, la información se agotó en las variables.

- Corr. OOF: correlación de sus predicciones fuera de fold con las de LightGBM; entre 0,98 y 0,99.
- AutoGluon sobre la base cruda, sin nuestras variables: 0,763.
- Mezclar los mejores suma una milésima (0,782).

El techo es común a todos: es de información, no de algoritmo.

Fuente: views/tablas/nivel7/etapa7_b.csv, etapa10_candidatos.csv, etapa12_resumen.csv, etapa18_resumen.csv.



Qué decir. Fuera de la lista del curso probamos CatBoost, redes neuronales tabulares y AutoML. Con nuestras variables todos quedan a menos de 0,015 de LightGBM y sus predicciones son casi idénticas. AutoGluon con la base cruda pierde 1,7 puntos: lo que le falta son nuestras variables, no capacidad de modelar.

Pregunta difícil: ¿por qué gana LightGBM y no la red? Con datos tabulares de este tamaño, los árboles con boosting suelen ganar a las redes; aquí la diferencia es de 0,005, del orden del efecto mínimo detectable.

Más complejidad compra 6 puntos de F1 y protege más a los más pobres

	Encuesta corta (Logit)	Logit completo	Mejor modelo
Variables	8	48	1.324 + auxiliares
F1 holdout	0,664	0,718	0,782
Subcobertura	26,0 %	23,2 %	18,2 %
Filtración	39,8 %	32,7 %	25,1 %
Excluidos en pobreza extrema	16,1 %	14,8 %	11,2 %
Interpretación	coeficientes	coeficientes	importancia y curvas
Costo de recolección	bajo	medio	alto (módulo laboral)

Pobreza extrema: ingreso menor a la mitad de la línea, como porcentaje de los pobres excluidos.

La complejidad se justifica por a quién deja de excluir, no por el F1 en sí.

Holdout, umbral elegido con OOF. Fuente: views/tablas/nivel7/etapa15_pesos_bienestar.csv (mejor modelo: mezcla sin el modelo limpio).

Más complejidad compra 6 puntos de F1 y protege más a los más pobres

Universidad de los Andes

	Encuesta corta (Logit)	Logit completo	Mejor modelo
Variables	8	48	1.324 + auxiliares
F1 holdout	0,664	0,718	0,782
Subcobertura	26,0 %	23,2 %	18,2 %
Filtración	39,8 %	32,7 %	25,1 %
Excluidos en pobreza extrema	16,1 %	14,8 %	11,2 %
Interpretación	coeficientes	coeficientes	importancia y curvas
Costo de recolección	bajo	medio	alto (módulo laboral)

Pobreza extrema: ingreso menor a la mitad de la línea, como porcentaje de los pobres excluidos.

La complejidad se justifica por a quién deja de excluir, no por el F1 en sí.

Holdout, semilla singular con ECAP. Fuente: scripts/baseline.jl (Etapa 0), scripts/ebm.jl (Etapa 3), scripts/interpretacion.jl (Etapa 4) y scripts/costo_recoleccion.jl (Etapa 5).

Qué decir. El trade-off concreto: pasar de 8 preguntas a un Logit completo y luego al mejor modelo baja la subcobertura de 26 a 18 % y la filtración de 40 a 25 %. Y no solo excluye menos: los que excluye están más cerca de la línea.

Pregunta difícil: ¿el mejor modelo es interpretable? No como un Logit, pero sus decisiones se explican con dos cosas: ingreso predicho y vivienda. Un EBM llega a 0,777 con curvas legibles, por si la agencia exige transparencia.

¿El modelo con más score en Kaggle? No necesariamente

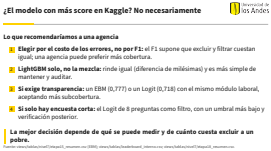
Universidad de los Andes

Lo que recomendaríamos a una agencia

1. **Elegir por el costo de los errores, no por F1:** el F1 supone que excluir y filtrar cuestan igual; una agencia puede preferir más cobertura.
2. **LightGBM solo, no la mezcla:** rinde igual (diferencia de milésimas) y es más simple de mantener y auditar.
3. **Si exige transparencia:** un EBM (0,777) o un Logit (0,718) con el mismo módulo laboral, aceptando más subcobertura.
4. **Si solo hay encuesta corta:** el Logit de 8 preguntas como filtro, con un umbral más bajo y verificación posterior.

La mejor decisión depende de qué se puede medir y de cuánto cuesta excluir a un pobre.

Fuente: views/tablas/nivel7/etapa15_resumen.csv (EBM); views/tablas/leaderboard_interno.csv; views/tablas/nivel7/etapa18_resumen.csv.



Qué decir. No necesariamente el de mayor Kaggle. La diferencia entre la mezcla y LightGBM solo es de milésimas, así que recomendamos el más simple. Si la agencia necesita explicar cada decisión, un EBM pierde medio punto. Y si solo puede pagar una encuesta corta, debe compensar con un umbral más bajo y verificación.

Pregunta difícil: ¿por qué no recomiendan directamente el Logit, que es transparente? Porque excluye a 5 puntos más de pobres y más de ellos en pobreza extrema; esa es una pérdida de bienestar que hay que poner en la balanza.

Glosario (1/2): métricas y errores

Precisión	De los hogares que el programa atiende, qué parte es pobre. 1−precisión = **filtración**: no pobres que reciben el beneficio.
Recall	De los hogares pobres, qué parte atiende el programa. 1−recall = **subcobertura**: pobres excluidos.
F1	Media armónica de precisión y recall: solo es alta si ambas lo son. No premia a los no pobres bien clasificados.
Umbral	Probabilidad a partir de la cual un hogar se clasifica como pobre. Bajarlo cubre más pobres a cambio de más filtración.
IC 95 %	Rango de valores compatibles con los datos. Si el IC de una diferencia incluye cero, no podemos afirmar que haya mejora.
Efecto mínimo detectable	La mejora más pequeña que el diseño detectaría con probabilidad 0,8.

Glosario (1/2): métricas y errores

Universidad de los Andes

Precisión	De los hogares que el programa atiende, qué parte es pobre. 1 − precisión ::: **filtración**: no pobres que reciben el beneficio.
Recall	De los hogares pobres, qué parte atiende el programa. 1 −recall ::: **subcobertura**: pobres excluidos.
F1	Media armónica de precisión y recall: solo es alta si ambas lo son. No premia a los no pobres bien clasificados.
Umbral	Probabilidad a partir de la cual un hogar se clasifica como pobre. Bajarlo cubre más pobres a cambio de más filtración.
IC 95 %	Rango de valores compatibles con los datos. Si el IC de una diferencia incluye cero, no podemos afirmar que haya mejora.
Efecto mínimo detectable	La mejora más pequeña que el diseño detectaría con probabilidad 0,8.

Glosario de apoyo; se muestra solo si lo preguntan.

Glosario (2/2): validación y variables

Holdout	20 % del train (32.993 hogares) apartado al inicio; solo se usa para medir, nunca para decidir.
Validación cruzada	El 80 % restante se parte en 5 bloques (folds): se entrena con 4 y se evalúa en el quinto, rotando.
Fuera de fold (OOF)	La predicción de cada hogar hecha por un modelo que no lo vio. Con ella se eligen umbral, pesos e hiperparámetros.
CV repetida	La validación cruzada repetida con 3 particiones nuevas: 15 evaluaciones para medir la variabilidad.
Auxiliar de ingreso	Modelo que predice el ingreso de cada persona con variables que el test sí trae; sumado al hogar, entra como variable.
OOF anidada	El auxiliar que alimenta a un hogar no vio ese hogar ni su fold de validación: así no se filtra la respuesta.

Glosario (2/2): validación y variables

Universidad de los Andes

Holdout	20 % del train (32.993 hogares) apartado al inicio; solo se usa para medir, nunca para decidir.
Validación cruzada	El 80 % restante se parte en 5 bloques (folds): se entrena con 4 y se evalúa en el quinto, rotando.
Fuera de fold (OOF)	La predicción de cada hogar hecha por un modelo que no lo vio. Con ella se eligen umbral, pesos e hiperparámetros.
CV repetida	La validación cruzada repetida con 3 particiones nuevas: 15 evaluaciones para medir la variabilidad.
Auxiliar de ingreso	Modelo que predice el ingreso de cada persona con variables que el test sí trae; sumado al hogar, entra como variable.
OOF amidada	El auxiliar que alimenta a un hogar no vio ese hogar ni su fold de validación: así no se filtra la respuesta.

Glosario de apoyo; se muestra solo si lo preguntan.

Apéndice: hiperparámetros del LightGBM ganador

Universidad de los Andes

Hiperparámetro	Rango explorado	Elegido	Lectura
Número de árboles	[200, 2000]	1.852	muchos árboles
Tasa de aprendizaje	$10^{[-3,\ -0{,}7]}$	0,011	pasos pequeños
Profundidad máxima	[3, 12]	12 (31 hojas)	árboles medianos
Mínimo de hogares por hoja	[10, 200]	171	no memoriza casos
Variables por corte	[0,2; 1]	0,82	algo de azar
Hogares por árbol	[0,5; 1]	0,70	algo de azar
Ganancia mínima para cortar	$10^{[-10,\ 1]}$	7×10^{-10}	sin restricción

Hipercubo latino de 50 configuraciones, validación cruzada de 5 folds sobre las 48 variables. Fuente: views/tablas/leaderboard_interno.csv (n6_lightgbm).

Apéndice: hiperparámetros del LightGBM ganador

Universidad de los Andes

Hiperparámetro	Rango explorado	Elegido	Lectura
Número de árboles	[200, 2000]	1.852	muchos árboles
Tasa de aprendizaje	$10^{[-1.3,-0.7]}$	0,011	pasos pequeños
Profundidad máxima	[3, 12]	12 (31 hojas)	árboles medianos
Mínimo de hogares por hoja	[10, 200]	171	no memorizar casos
Variables por corte	[0,2; 1]	0,82	algo de azar
Hogares por árbol	[0,5; 1]	0,70	algo de azar
Ganancia mínima para cortar	$10^{[-10,-1]}$	7×10^{-10}	sin restricción

Hiperparámetros de la configuración ganadora, utilizada en cada fecha sobre las 16 variables. Fuente: notebooks/lightgbm/lightgbm_40mensuales_16f_lightgbm.

Respaldo: la configuración que se reutilizó en todo el nivel 7.

Apéndice: en CV repetida ninguna alternativa mejora de forma detectable

Candidato	F1 CV repetida (EE)	Δ vs. LightGBM [IC 95 %]	F1 holdout [IC 95 %]
LightGBM base	0,7752 (0,0008)		0,7806 [0,7731; 0,7885]
+ redes (mezcla sin limpio)	0,7758 (0,0008)	+0,0006 [−0,0001; +0,0013]	0,7820 [0,7743; 0,7897]
+ redes + limpio (final)	**0,7759 (0,0008)**	+0,0007 [+0,0002; +0,0012]	0,7817 [0,7742; 0,7894]
CatBoost	0,7722 (0,0007)	−0,0030 [−0,0042; −0,0018]	0,7786 [0,7707; 0,7864]
Rediseño desde cero	0,7478 (0,0008)	−0,0274 [−0,0289; −0,0260]	0,7499 [0,7420; 0,7581]

IC pareados ingenuos (15 pares). Con la corrección de Nadeau y Bengio el IC del modelo final es [−0,0004; +0,0017]. Fuente: views/tablas/nivel7/etapa16_fase1_resumen.csv, etapa18_resumen.csv.

Apéndice: en CV repetida ninguna alternativa mejora de forma detectable

Candidato	F1 CV repetida (SE)	Δ vs. LightGBM [IC 95 %]	F1 holdout [IC 95 %]
LightGBM base	0,7752 (0,0008)		0,7804 [0,7731; 0,7865]
+ redes (exacta sin limpia)	0,7758 (0,0008)	+0,0006 [−0,0001; +0,0013]	0,7828 [0,7743; 0,7897]
+ redes + limpia (final)	**0,7759 (0,0008)**	+0,0007 [+0,0002; +0,0012]	0,7817 [0,7742; 0,7894]
CatBoost	0,7722 (0,0007)	−0,0030 [−0,0042; −0,0018]	0,7786 [0,7707; 0,7864]
Rediseño desde cero	0,7478 (0,0008)	−0,0274 [−0,0289; −0,0260]	0,7499 [0,7420; 0,7581]

CV: permutación, regresora 10 pasos. Corrida exacta: 2 of 5a corrección de Nadeau y Bengio 457 del modelo final en [−0.0002, +0.0017]. Fuentes: runway/tables/model3/stage20_base2_resumidas.csv, stage22_resumen.csv.

Respaldo: CatBoost pierde con un IC que excluye cero; la mezcla gana por milésimas.